Trabalho de Sistemas Distribuídos

Prof. Alessandro Vivas Andrade

Objetivo

Implementar um sistema distribuído para cadastro, submissão, avaliação e publicação de monografias (TCC) do curso, permitindo que alunos submetam seus trabalhos, orientadores acompanhem e aprovem a submissão, coordenadores agendem bancas de defesa, e o público em geral pesquise monografias aprovadas em um repositório digital.

O sistema deverá garantir que a submissão para banca ocorra somente após aprovação do orientador, validar a integridade dos arquivos enviados (formato, tamanho, hash criptográfico), e publicar automaticamente no repositório público apenas monografias aprovadas em defesa e com o termo de autorização de publicação assinado.

Além disso, o sistema deverá ser projetado para suportar milhares de acessos simultâneos ao repositório público — especialmente em períodos de defesa de TCC —, utilizando técnicas modernas de sistemas distribuídos, incluindo cache Redis, filas de eventos (RabbitMQ ou Kafka) e arquitetura baseada em microsserviços.

Requisitos

- Django (última versão)
- Banco de Dados PostgreSQL
- Python
- Django REST Framework
- Redis (cache e controle de submissões)
- RabbitMQ ou Kafka (fila de eventos)
- Biblioteca para leitura e extração de texto de PDF (ex.: PyPDF2, pdfplumber)
- Frontend em JavaScript
- Grupos de até 05 alunos
- Linux

Parte 1: Autenticação, Submissão e CRUD

Autenticação

Implementação de um sistema de autenticação utilizando django-allauth.

O sistema deve possuir:

- Login e senha para acesso ao sistema
- Controle de acesso baseado em permissões para diferentes tipos de usuários:
  - Administradores
  - Coordenadores de curso
  - Professores/Orientadores
  - Alunos
  - Membros de banca externos (acesso temporário por token, sem conta permanente)
- Criptografia de senhas
- Políticas de senha seguras
- Registro de auditoria para rastrear as ações dos usuários
- Página principal do sistema
- Dashboard personalizado por tipo de usuário

Modelos Principais

Sugestão de estrutura de dados.

- **Aluno:** nome, matrícula, email, curso, orientador atual
- **Professor:** nome, email, departamento, área de pesquisa
- **Curso:** nome, código, coordenador
- **Monografia:** título, resumo, palavras-chave, aluno, orientador, curso, data de submissão, arquivo (PDF), hash do arquivo (SHA-256), status (rascunho / em avaliação / aprovada / reprovada / publicada)
- **Banca:** monografia, data, local ou link (para defesas remotas), membros (professores e/ou externos), status (agendada / realizada / cancelada)
- **Avaliação:** banca, membro avaliador, nota, parecer, recomendação (aprovado sem ressalvas / aprovado com ressalvas / reprovado)
- **Documento Complementar:** monografia, tipo (ata de defesa, termo de autorização de publicação, comprovante de correção ortográfica), arquivo

Submissão de Monografia

O aluno poderá:

- criar um rascunho da monografia e associar um orientador
- fazer upload do arquivo em PDF
- acompanhar o status da submissão

O sistema deve validar, no upload:

- formato do arquivo (somente PDF)
- tamanho máximo do arquivo (ex.: 50 MB)

- cálculo do hash SHA-256 do arquivo, para deduplicação e verificação de integridade

Somente após a aprovação do orientador a monografia pode ser encaminhada para agendamento de banca. Sem essa aprovação, o agendamento não deve ser permitido.

A extração de texto do PDF (para indexação de busca) deve ser feita de forma assíncrona, via fila de eventos — não durante a requisição de upload.

Agendamento de Banca

O coordenador ou o orientador poderá:

- agendar uma banca de defesa para uma monografia já aprovada pelo orientador
- definir data, local (presencial) ou link (defesa remota)
- convidar membros da banca, incluindo a possibilidade de um membro externo (acesso via token temporário, sem necessidade de conta permanente no sistema)

O sistema deve validar:

- se a monografia já foi aprovada pelo orientador
- se todos os membros convidados confirmaram disponibilidade
- se não há conflito de horário com outras bancas dos mesmos membros

Após o agendamento, o sistema deve notificar automaticamente (via fila de eventos) todos os membros da banca e o aluno.

Após a defesa, cada membro da banca registra sua avaliação (nota e parecer). O sistema deve calcular o resultado final somente após todas as avaliações serem registradas, e então atualizar o status da monografia.

Publicação no Repositório

Após aprovação final em banca, envio da versão corrigida (quando houver ressalvas) e assinatura do termo de autorização de publicação, a monografia deve ser publicada automaticamente no repositório público.

O repositório público deve ser pesquisável por: título, autor, orientador, curso, ano de defesa e palavras-chave.

Verificação de Integridade e Controle de Acesso

O sistema deve armazenar o hash SHA-256 de cada arquivo submetido, permitindo detectar qualquer alteração não autorizada após a aprovação.

Ações administrativas e de submissão (upload, aprovação de orientador, agendamento de banca, lançamento de avaliação) devem ser restritas a usuários autenticados, e a área administrativa do sistema deve validar que o acesso ocorre a partir da rede institucional (IP/VPN).

A leitura pública do repositório de monografias publicadas, por outro lado, deve ser aberta a qualquer visitante, sem exigir autenticação.

CRUD

O sistema deve permitir operações CRUD para:

- alunos
- professores
- cursos
- monografias
- bancas
- avaliações

Funcionalidades obrigatórias:

- **Criação:** cadastro de dados e upload de arquivos
- **Leitura:** busca por aluno, orientador, curso, ano de defesa ou palavras-chave
- **Atualização:** permitir edição de monografias em rascunho, bancas ainda não realizadas, e avaliações (somente pelo respectivo avaliador, antes do fechamento do resultado)
- **Exclusão:** permitir exclusão apenas para usuários autorizados, e nunca de monografias já publicadas no repositório

Parte 2: REST API com Django REST Framework

O sistema deve disponibilizar uma API REST.

Endpoints Públicos

- lista de monografias publicadas (com busca e filtros)
- lista de cursos
- lista de orientadores

Formato de resposta: JSON.

Endpoints Restritos

CRUD autenticado para:

- alunos
- monografias
- bancas
- avaliações

Exemplos:

```
POST /api/monografias
POST /api/bancas
GET  /api/monografias?busca=sistemas+distribuidos
```

Arquitetura de Microsserviços

O sistema deverá ser dividido em múltiplos serviços independentes, por exemplo:

- **Serviço de Autenticação:** login, controle de usuários e permissões
- **Serviço de Monografias:** CRUD, upload de arquivos, cálculo de hash
- **Serviço de Bancas:** agendamento, avaliação, cálculo de resultado final
- **Serviço de Repositório e Busca:** indexação full-text e busca pública
- **Serviço de Notificações:** envio de e-mails e alertas
- **Serviço de Relatórios:** estatísticas e dashboards

Cada serviço deverá possuir sua própria API.

Uso de Redis (Cache)

Redis deverá ser utilizado para:

- cache de sessões
- cache de resultados de busca no repositório
- controle de uploads em processamento (evitando reenvio duplicado enquanto a extração de texto ainda está em andamento)
- limitação de taxa de submissões (rate limiting)

Exemplo de chave:

```
busca:hash_da_query
upload_em_andamento:monografia_id
```

Fila de Eventos (RabbitMQ ou Kafka)

O sistema deverá utilizar uma fila de mensagens para processar eventos de forma assíncrona.

Eventos possíveis:

- submissão de monografia (dispara extração de texto e indexação)
- agendamento de banca (dispara notificações aos membros e ao aluno)
- avaliação registrada (dispara verificação se todas as avaliações da banca já foram lançadas, e cálculo do resultado)
- publicação (dispara indexação no repositório público e notificação por e-mail ao aluno)

Fluxo exemplo:

Aluno submete monografia → evento enviado para a fila → serviço extrai o texto do PDF e indexa → serviço notifica o orientador.

Isso permite escalabilidade e desacoplamento entre os serviços.

Escalabilidade

O sistema deve ser projetado para suportar milhares de visitantes pesquisando o repositório simultaneamente, especialmente em períodos de defesa de TCC.

Estratégias recomendadas:

- cache Redis para resultados de busca frequentes
- processamento assíncrono com filas para tarefas pesadas (extração de texto, indexação)
- serviços independentes
- APIs desacopladas

Dashboard

Criar um dashboard com gráficos contendo:

- monografias por curso
- monografias por orientador
- monografias por ano de defesa
- taxa de aprovação
- tempo médio entre submissão e defesa

Ferramentas sugeridas: Chart.js ou Plotly.

Frontend

Criar interface web utilizando JavaScript. A interface deve permitir:

- submissão de monografia pelo aluno
- acompanhamento e aprovação pelo orientador
- agendamento de banca pelo coordenador
- lançamento de avaliação pelos membros da banca
- pesquisa pública no repositório de monografias

O frontend deve consumir a API do sistema.

Histórico de Alterações

O sistema deve registrar:

- quem realizou cada alteração
- qual alteração foi realizada
- data e hora da alteração